WADZZO · DEPLOYMENT.MD

Deployment

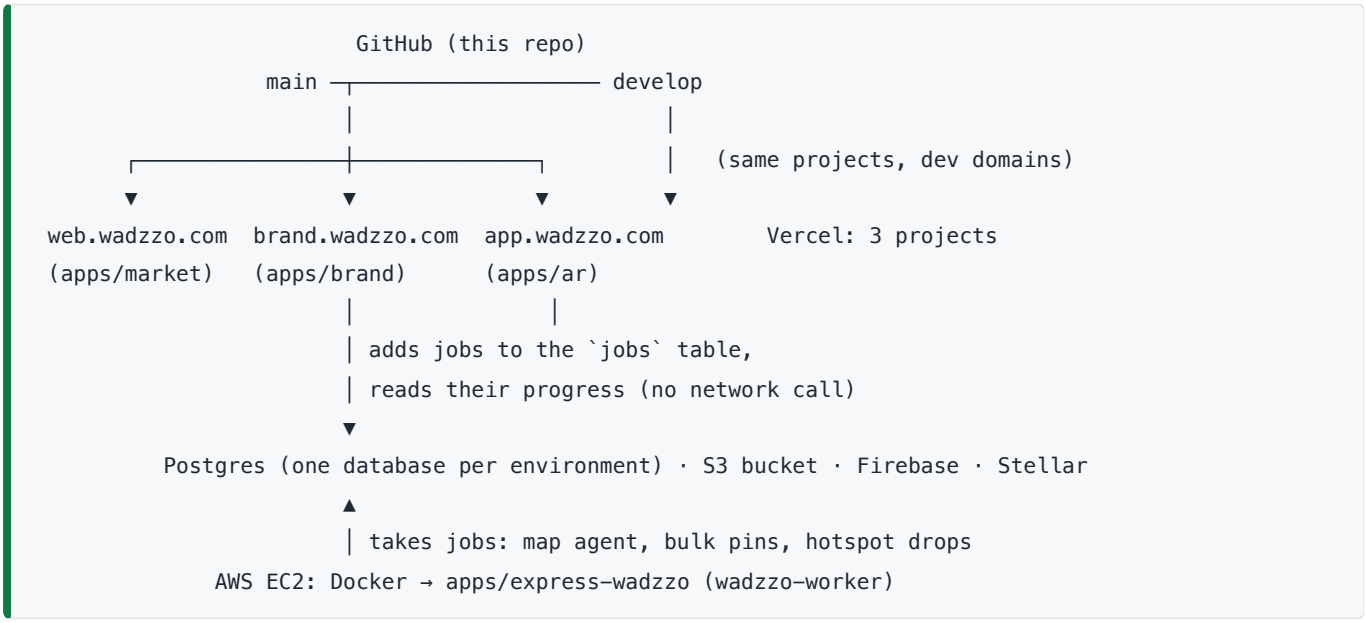
Generated 2026-11-07 from docs/DEPLOYMENT.md · 299 lines of source

Contents

1. Environments
2. Vercel: one project per app
3. Database
4. The worker on AWS
5. First deployment, in order
6. Releases
7. White-label platforms (Clinton County)
8. Troubleshooting

What this guide does not cover

The three web apps run on **Vercel**, the worker runs on **AWS (EC2)**. Everything is deployed from this one repository.



Piece	Where	Deployed by
Fan marketplace	Vercel project `wadzzo` (`apps/market`)	Vercel Git integration
Brand portal	Vercel project `brand-wadzzo` (`apps/brand`)	Vercel Git integration
AR collector + mobile API	Vercel project `ar-web-wadzzo` (`apps/ar`)	Vercel Git integration
Clinton County brand portal	Vercel project `brand-clinton` (`apps/brand`)	Vercel Git integration
Clinton County AR collector	Vercel project `clinton-ar` (`apps/ar`)	Vercel Git integration
Worker	AWS EC2, Docker (`wadzzo-worker`)	`.github/workflows/deploy-express-docker.yml`
Mobile app	App Store / Play (Expo)	EAS, outside this guide

All five Vercel projects (team `wadzzo2023s-projects`) are connected to this repository; each one's Root Directory picks the app, and `PLATFORM_SLUG` picks the platform. One push rebuilds only the projects whose app or packages changed

(`ignoreCommand` in each `vercel.json`).

1. Environments

Environment	Git branch	Web apps (domains)	Network	Database	Worker
Production	`main`	`web.wadzzo.com` , `brand.wadzzo.com` , `app.wadzzo.com`	mainnet	production Postgres	AWS EC2 (this guide)
Development	`develop`	`dev-web.wadzzo.com` , `dev-brand.wadzzo.com` , `dev-app.wadzzo.com`	testnet	development Postgres	a development worker, or none
Preview	any other	`*.vercel.app` (one per push)	testnet	development Postgres	none: jobs queue up and nothing answers (see 4.4)
Local	-	`localhost:3000/3001/3002`	testnet	`pnpm db:local` or a dev DB	`pnpm --filter @wadzzo/express dev`

The `dev-*` names are a proposal (they need no wildcard DNS and sit under the apex you already own); pick another scheme if you prefer, the steps are the same.

Rules that keep the environments apart:

- A deployment is **one network**: `NEXT_PUBLIC_STELLAR_NETWORK` is `mainnet` in Production and `testnet` everywhere else. There is no runtime switch.
- The three apps of one environment share **one database, one bucket, the same platform wallets and the same issuer key**. Different environments share nothing.
- `ISSUER_ENCRYPTION_KEY` of Development must be a different key from Production's. It must be identical in the three apps of its own environment.
- On Vercel every deployment, previews included, runs with `NODE_ENV=production` . The code therefore uses `VERCEL_ENV` to decide what "production" means.

2. Vercel: one project per app

2.1 Create the projects (three times)

Vercel → **Add New…** → **Project** → import this repository, then:

Setting	`wadzzo`	`brand-wadzzo`	`ar-web-wadzzo`
Framework preset	Next.js	Next.js	Next.js
Root Directory	`apps/market`	`apps/brand`	`apps/ar`
Include source files outside of the Root Directory	on	on	on
Node.js Version	Vercel's default (22.x)	same	same
Production Branch	`main`	`main`	`main`

The remaining behaviour is committed in each app's `vercel.json` : build command (`pnpm run build` , which runs `sync:platform` first), the ignored-build step (a push that doesn't touch the app or a package it depends on does not rebuild it), `framework` , and a 60 s function limit.

Add this environment variable to **all three** projects (Production, Preview, Development), so Vercel uses the pnpm version pinned in `package.json`:

```
ENABLE_EXPERIMENTAL_COREPACK=1
```

Check the first build log for `pnpm 12.x`.

2.2 Domains

Per project → **Settings** → **Domains**:

Project	Production domain	`develop` branch domain
`wadzzo`	`web.wadzzo.com`	`dev-web.wadzzo.com`
`brand-wadzzo`	`brand.wadzzo.com`	`dev-brand.wadzzo.com`
`ar-web-wadzzo`	`app.wadzzo.com`	`dev-app.wadzzo.com`
`brand-clinton`	`clintonadmin.wadzzo.com`	
`clinton-ar`	`clinton.wadzzo.com`	

Add each domain, then for the dev ones choose **Git Branch** → `develop`. At your DNS provider create the record Vercel shows for each (a CNAME for subdomains). `www.wadzzo.com` should redirect to `web.wadzzo.com` (add it in the market project with "Redirect to").

Create the `develop` branch once: `git switch -c develop && git push -u origin develop`.

2.3 Environment variables

Each app's `apps/<app>/.env.example` lists every variable it reads, grouped, with what it is for. Enter them in **Settings** → **Environment Variables**:

- Values for Production → environment **Production**.
- Development values → environment **Preview**, scoped to the `develop` branch.
- Anything left on plain "Preview" (other branches) should hold only values that are safe for a throwaway deployment, and must not hold `NEXTAUTH_URL` (NextAuth then falls back to the preview's own URL).

Quickest way: `vercel env add`, or paste a whole `.env` block into the dashboard's "Import .env" (it accepts the sectioned files in `apps/<app>/.env.production`).

Values that differ between Production and Development:

Variable	Production (`main`)	Development (`develop`)
`PLATFORM_SLUG`	`wadzzo`	`wadzzo`
`NEXT_PUBLIC_URL`, `NEXTAUTH_URL`	the app's production domain	the app's `dev-*` domain
`NEXT_PUBLIC_STELLAR_NETWORK`	`mainnet`	`testnet`
`NEXT_PUBLIC_STAGE`	`prod` (real network and platform fees)	`dev`
`NEXT_PUBLIC_ASSET_CODE` / `_ISSUER`	the mainnet platform token	a testnet token
`NEXT_PUBLIC_HOME_DOMAIN`	`web.wadzzo.com`	`dev-web.wadzzo.com`
`MOTHER_SECRET`, `STORAGE_SECRET`	mainnet accounts	funded **testnet** accounts
`ISSUER_ENCRYPTION_KEY`	production key	**a different key**

Variable	Production (`main`)	Development (`develop`)
`DATABASE_URL`	production Postgres	development Postgres
`DATABASE_POOL_MAX`	`1` (see section 3)	`1`
`AWS_BUCKET_NAME` and keys	production bucket	a separate bucket
`NEXTAUTH_SECRET`	production secret	a different secret
`SQUARE_*` , `NEXT_PUBLIC_SQUARE_*`	production credentials	**sandbox** credentials
`NEXT_PUBLIC_WADZZO_AR_URL` (brand)	`https://app.wadzzo.com`	`https://dev-app.wadzzo.com`
`NEXT_PUBLIC_BRAND_PORTAL_URL` (market)	`https://brand.wadzzo.com`	`https://dev-brand.wadzzo.com`
`BRAND_APP_ORIGINS` (ar)	`https://brand.wadzzo.com`	`https://dev-brand.wadzzo.com`

Variables that must be **identical across the three apps of one environment**: `DATABASE_URL` , `ISSUER_ENCRYPTION_KEY` , `MOTHER_SECRET` , `STORAGE_SECRET` , `RUBY_ACCOUNTS_SECRET` , the asset code/issuer, the network, the bucket and its keys, the Firebase project and admin key, `PLATFORM_SLUG` . The worker's `DATABASE_URL` must be the same one.

Before you change anything in a dashboard, run the same check against your files: `pnpm env:check` .

2.4 Third-party allow-lists (the usual reasons a deploy "works but can't log in")

Add the six domains (three production, three `dev-*`) to:

Service	Where	Why
Firebase Authentication	Console → Authentication → Settings → **Authorized domains**	Google / Apple / Facebook popup sign-in
Sign in with Apple	Apple developer → Services ID → domains and return URLs	Apple sign-in on the web
Facebook login	App settings → valid OAuth redirect URIs	Facebook sign-in
WalletConnect	Cloud project → allowed domains	WalletConnect pairing
Square	Application → Web Payments allowed domains (market only)	card payments
Mapbox / Google Maps	token / key URL restrictions	maps
Stellar `home_domain`	the issuer account must point at `NEXT_PUBLIC_HOME_DOMAIN`	`stellar.toml` at `https://<home domain>/.well-known/stellar.toml` is served by the market app

2.5 What a Vercel deploy does

1. Install at the repository root (`pnpm install` , which also runs `prisma generate` through `@wadzzo/db` 's `postinstall`).
2. `pnpm run build` in the app: `sync:platform` (theme and assets for `PLATFORM_SLUG`), then `next build` .
3. Functions are created per route. The AR project ships the `onnxruntime-node` Linux binaries it needs (`outputFileTracingIncludes` in `apps/ar/next.config.js`).

Pushes to `main` promote to Production; pushes to `develop` update the `dev-*` domains. **Branch protection** on `main` and `develop` should require the `CI` check (`ci.yml`) so a red build can't deploy.

3. Database

- **One database per environment**, shared by the three apps and the worker.
- **Production runs on the EC2 Postgres** (`DATABASE_URL` host in your production files). Vercel functions have no fixed IP addresses, so the database's security group must accept port 5432 from Vercel (open it with TLS enforced and a strong password, or use Vercel's Static IPs if your plan includes them).
- **Connection count.** Serverless functions each hold their own connection. Set `DATABASE_POOL_MAX=1` on the Vercel projects and put **PgBouncer** (transaction mode) in front of Postgres, or Postgres's `max_connections` will be reached under load. The worker connects directly and may keep the default pool.
- **Schema changes** live only in `packages/db/prisma/schema.prisma`. Apply them from a trusted machine, never from a build:

  ```
  CONFIRM_DB_PUSH=yes DATABASE_URL="<env's url>" pnpm --filter @wadzzo/db push
  ```

  Order for a release that changes the schema: apply the (additive) schema change to the database first, then let the apps deploy. For a breaking change, deploy the code that no longer needs the old column first, then drop it.
- Development uses its own Postgres (Neon works well: its pooler endpoint is already pooled).

4. The worker on AWS

The brand apps never call the worker. They add a row to the `jobs` table and read the result from the same table; the worker (`apps/express-wadzzo`) takes the rows and does the work: the **map agent** (`agent_run`), **bulk pin creation** (`create_pins`) and **hotspot drops** (`hotspot_drop` , added every 5 minutes). Because the database is the only link, there is no worker address to set, no shared secret, and no CORS; and a job survives a restart.

Current production: one Docker container, `wadzzo-worker` , on the `bandcoin-worker` EC2 instance (us-west-2, Elastic IP `54.69.28.110`), next to the Bandcoin worker. Its only HTTP route is `/health` , behind Caddy at `https://wadzzo.54-69-28-110.sslip.io/health` (the host and the watchdog use it; it returns 503 if the worker's loop is stuck).

4.1 How a deploy works

`.github/workflows/deploy-express-docker.yml` runs on a push to `main` that touches the worker or a package it bundles, when the repository variable `DEPLOY_EXPRESS_DOCKER` is `true` :

1. Tests the worker, `@wadzzo/jobs` and `@wadzzo/geo` .
2. Builds the image on the runner and streams it to the server (the server never compiles).
3. **Creates the `jobs` table if it is missing** (`scripts/apply-sql.sh` with `packages/db/prisma/sql/2026-11-07-job-queue.sql` : additive, safe to run again, uses the server's own env file, so no database password is stored in GitHub).
4. `scripts/deploy-express.sh` swaps the container: it asks the old one to stop (it finishes the job in hand, up to 120 s), starts the new one, waits for `/health` , and goes back to the previous image if it never answers.

Secrets: `EC2_SSH_KEY` , `EC2_HOST` , `EC2_USER` . Run it by hand with **stage_only** to load an image without starting it. The server's env file is `~/wadzzo/express.env` (never in git).

Switching the brand app over (the release that introduced the queue). Vercel builds the brand app on the same push, usually before the worker image is ready. The brand app needs the `jobs` table the moment it is live, so apply it **before** that push:

```
scp -i <key> packages/db/prisma/sql/2026-11-07-job-queue.sql scripts/apply-sql.sh
ubuntu@<host>:wadzzo/
ssh -i <key> ubuntu@<host> 'bash ~/wadzzo/apply-sql.sh ~/wadzzo/2026-11-07-job-queue.sql'
```

Chat messages and pin batches sent between the brand app going live and the new worker starting simply wait in the queue and are done once it is up. Deploy when no pin batch is running.

4.2 Environment

The worker's env file needs `DATABASE_URL` (the **same database as the brand apps of this environment**: that is how the two find each other), `OPENAI_API_KEY`, `GOOGLE_MAP_API_KEY`, the `AWS_*` bucket values, and optionally `JOB_TIMEOUT_MS` (default 300000) and `WORKER_CONCURRENCY` (default 5). `NEXTAUTH_SECRET`, `ALLOWED_ORIGINS` and `RATE_LIMIT_*` are no longer used and can be removed. On Vercel, `EXPRESS_SERVER_URL` is no longer used and can be removed.

4.3 Operating it

Task	Command (on the server)
Logs	`docker logs -f wadzzo-worker`
Status, memory	`docker ps`, `curl -s http://127.0.0.1:4000/health`
Queue, by state	`/health` → `jobs` (`queued`, `running`, `succeeded`, `failed`)
Restart	`docker restart wadzzo-worker` (it finishes the job in hand first)
Roll back	revert the commit on `main`; the workflow redeploys. The script also goes back itself if the new image never answers

A job whose worker died is put back (or, for a job meant to run once such as pin creation, marked failed and never run twice) after 5 minutes without a heartbeat. A pin batch card that stops moving shows as failed after 10 minutes and says what was made.

4.4 Other environments

A job is picked up by whichever worker uses the same database, so **each environment needs its own worker on its own database**: without one, the map agent and bulk pins queue up and nothing answers (hotspot create, pause and delete still work, they are database writes). For development, run `pnpm --filter @wadzzo/express dev` against the development database; a second container with the development `DATABASE_URL` on the server works too. A worker on the production database never sees development jobs, and the reverse.

5. First deployment, in order

1. Database ready and schema applied (section 3). Buckets and Firebase projects exist.
2. The worker on AWS (section 4) and `/health` returns `ok`.
3. Create the three Vercel projects (2.1), add environment variables (2.3), domains (2.2), allow-lists (2.4).
4. Push `develop`; open the three `dev-*` domains and run the checks below.
5. Merge `develop` into `main`; open the three production domains and run the checks again.

Checks after any deployment

Check	Expect
`https://web.wadzzo.com/.well-known/stellar.toml`	the platform's organisation and currencies
`https://web.wadzzo.com` and `https://brand.wadzzo.com`	pages render with the right branding
`https://app.wadzzo.com/api/mobile/v1/bounties`	`200` JSON
Sign in with email, then with a wallet	a session, no redirect loop
Brand portal → create a drop with an image	the upload succeeds (bucket + keys)
Brand portal → map agent	a job is added and its steps appear (worker + jobs table)
An unauthenticated tRPC call that passes `signWith`	`401 AUTH_REQUIRED`

6. Releases

```
feature branch —PR→ develop —(verify on dev-* domains)→ main → production
```

- **Web apps:** Vercel keeps every deployment. **Rollback** is "Promote to production" on the previous deployment (instant, no rebuild).
- **Worker:** revert on `main` and let the workflow run, or run the workflow manually (Run workflow) with an older commit selected.
- **A change touching both** (for example, a new job type the brand app adds): deploy the worker first, so it knows the type, then the app. A job type the worker doesn't know just waits in the queue.

7. White-label platforms (Clinton County)

A partner is **the same code with another** `PLATFORM_SLUG`, not another branch. For each partner app create one more Vercel project from the same repository and Root Directory:

- `PLATFORM_SLUG=clintoncounty` (this picks the theme, assets and branding at build time).
- The partner's own domain, token, wallets and `TOML_ORG_*`.
- Only what differs is in `apps/<app>/.env.clintoncounty.production`; everything else is the Wadzzo project's value.
- The partner shares the Wadzzo database and worker (rows are scoped by platform).

See PLATFORMS.md for adding a platform.

8. Troubleshooting

Symptom	Cause and fix
Runtime error `ERR_REQUIRE_ESM … jose … jwks-rsa` (from `firebase-admin/auth`)	`firebase-admin` → `jwks-rsa@4` → `jose@6` is ES-modules-only and fails as an *external* module on a Node that cannot `require()` ES modules. Each app's `next.config.js` lists `firebase-admin`, `jwks-rsa` and `jose` in `transpilePackages`, so Next bundles them and the Node version doesn't matter. Keep those three entries (and don't pin `jwks-rsa` back to 3.x).
Build fails with `CONFIG_MISSING` / `Invalid environment variables`	A required variable is missing for that environment (Production vs Preview). Compare with `.env.example`.

Symptom	Cause and fix
Login popup closes, or "unauthorized domain"	The domain isn't in Firebase's authorized domains (2.4).
Login works but the session is lost on the next request	`NEXTAUTH_URL` differs from the domain being used, or `NEXTAUTH_SECRET` differs between deployments.
The map agent or a pin batch never answers	No worker is running on that environment's database, or it uses a different `DATABASE_URL` (section 4.4). `/health` on the worker shows the queue.
Uploads fail with `CONFIG_MISSING` naming `AWS_*`	Bucket variables missing. They are read as `AWS_*` or the older `NEXT_AWS_*`.
`too many connections` in Postgres logs	`DATABASE_POOL_MAX` not `1` on Vercel, or no PgBouncer (section 3).
A function times out at 60 s	Raise `maxDuration` in the app's `vercel.json` (plan limits apply), or move the work to the worker as a job.
The AR project's function is too large	Check the build output for the `onnxruntime-node` / `@huggingface/transformers` bundle sizes; the AR app was sized for these.
`pnpm env:check` reports a mismatch	Fix the value in the file *and* in the Vercel project; the check reads files, Vercel holds the truth.

What this guide does not cover

- EAS builds and store releases for `apps/mobile`.
- Backups and point-in-time recovery of Postgres.
- Cost sizing of Vercel plans (function duration and bandwidth limits depend on your plan; check them against the `maxDuration` values in `vercel.json`).